

PART II — BUILDING THE INGESTION PIPELINE

Chapter 4

The Document Structure

“The most fundamental units of any system are the ones you stop noticing — until the day they betray you.”

Chapter 4 — The Document Structure

Chapter 3 left you with a working environment. Every package imports, every key loads, every smoke-test prints its tick. Now the data starts moving — and the first object it moves through, the unit on which the entire ingestion pipeline is built, is the Document. Understanding it well is non-negotiable. Misunderstand it, and every chapter that follows will quietly leak bugs that look like retrieval failures but are really data-modelling failures dressed up in similarity scores.

This chapter is short on lines of code and long on consequences. The Document object has only two fields. That is all. Yet how you populate those two fields — particularly the second one — determines whether your retriever can cite sources, whether your filters work, whether your self-learning loop knows what to forget, and whether a teammate can audit your system six months from now. By the time you finish Section 4.4, you will see metadata not as a place to dump optional notes but as the connective tissue of the entire system.

The structure is straightforward. Section 4.1 defines what a Document object actually is. Section 4.2 separates its two fields and explains why one of them is the silent workhorse of every later chapter. Section 4.3 walks through how to enrich metadata in practice — what to add, where, and when. Section 4.4 closes the loop by tracing five concrete dividends that clean metadata pays the moment your project leaves the toy stage.

4.1 What a Document Object Really Is

Every loader in LangChain — every PDF parser, every CSV reader, every web scraper, every database adapter — returns the same data type. That uniformity is not an accident. It is the single most important design choice in the whole framework. Whatever the source format, whatever the file extension, the loader's job is to produce one or more instances of a single class: Document.

Definition — Document

An immutable, two-field data container defined in `langchain_core.documents`. It holds (1) `page_content` — a string carrying the textual payload — and (2) `metadata` — a dictionary carrying everything you know about that payload that is not the payload itself. Every component downstream of a loader operates on a list of these objects.

That is the entire surface area. No subclass per file format. No specialised `PDFDocument` or `CSVDocument` or `HTMLDocument`. The deliberate simplicity is what makes a splitter, an embedder, or a vector store completely indifferent to whether the text it received originated in a scanned PDF, a Markdown blog post, or a row of a SQL table. They see a Document; they handle a Document.

In code, instantiating one is exactly as plain as it sounds:

```
from langchain_core.documents import Document

doc = Document(
    page_content="In 1928, Alexander Fleming noticed that a "
                "Petri dish of Staphylococcus had been "
                "contaminated by a mould...",
    metadata={
        "source": "fleming_1929.pdf",
```

```

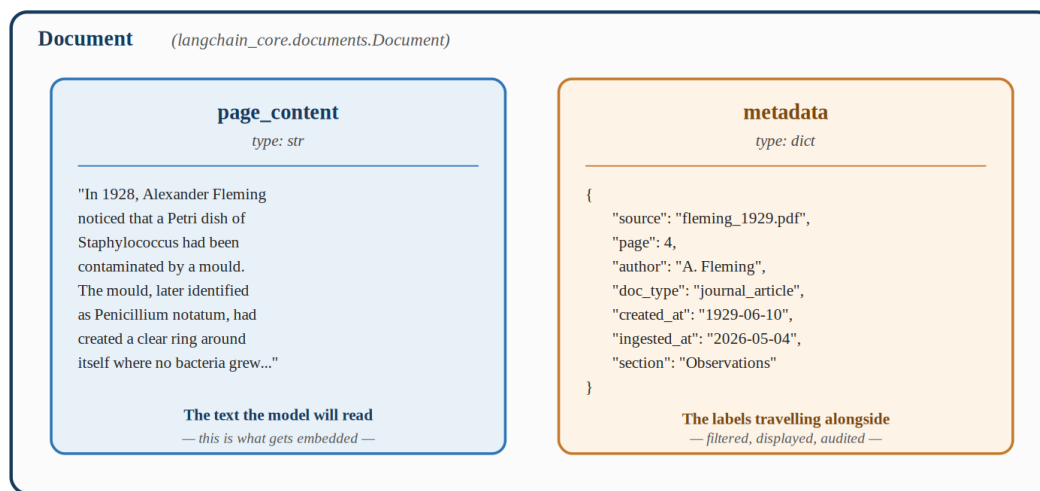
    "page": 4,
    "author": "A. Fleming",
    "doc_type": "journal_article",
  },
)

print(type(doc).__name__) # -> Document
print(doc.page_content[:30]) # -> In 1928, Alexander Fleming no
print(doc.metadata["source"]) # -> fleming_1929.pdf

```

The Anatomy of a Document Object

A two-field container: payload + provenance



Every Document carries two things: the words to retrieve on, and the labels that explain where those words came from.

Figure 4.1 — The anatomy of a Document object. A two-field container holding the textual payload (*page_content*) and a free-form dictionary of provenance and labels (*metadata*). Every loader in the pipeline emits this exact shape.

Analogy — The padded envelope

Think of a Document as a padded envelope in the postal system. Inside is a letter — that is *page_content*. On the outside is the address label, the postmark, the return address, the tracking barcode — that is *metadata*. The post office (your retriever) sorts and routes purely by what is on the outside. The recipient (the LLM) cares only about what is inside. Both halves are mandatory; an envelope without an address gets lost, and an addressed envelope with no letter is a wasted trip.

Two practical consequences follow from this simplicity, and both will matter later. First, because Document is just a container, you can construct one yourself anywhere — you do not need a loader. When you are testing a splitter or debugging a retriever, instantiating a handful of Document objects directly with literal strings is perfectly idiomatic and is what your unit tests will use. Second, because the class itself does not validate metadata, your loader and your code are responsible for putting the right keys in the right places. Nothing will scream when an important key is missing; the failure will surface much later as a missing citation or an empty filter, and that is precisely the failure mode the rest of this chapter is designed to help you avoid.

4.2 *page_content* vs *metadata* — and Why Metadata Matters

It is tempting, when staring at a Document for the first time, to treat the two fields as having equal weight. They do not. They serve fundamentally different masters and travel through the pipeline along different tracks. Conflating them is the most common conceptual error new RAG developers make, and the cost of that error compounds at every stage.

page_content — the searchable payload

page_content is the text the retriever will search over. It is the only thing the embedder converts into a vector. It is the only thing the LLM ultimately reads as context. If a fact is not in page_content, the system effectively does not know it — no matter how rich the surrounding metadata is. That is the first rule, and it is absolute: anything you want to be found by similarity search has to live in page_content.

This has a corollary that catches almost everyone the first time. Suppose you load a CSV row and store the row's ID in metadata but only the description in page_content. A user query like “what is record 4172's status?” will retrieve nothing relevant, because “4172” is not embedded; it lives in metadata, which is invisible to the embedder. If you want lookups by ID to work, the ID must also appear inside page_content — even if redundantly. Embedding does not see metadata. Repeat that sentence until it is reflexive.

metadata — the labels travelling alongside

metadata is a dictionary of arbitrary key/value pairs. Concretely, it is a Python dict with string keys and (in ChromaDB's case) primitive values: strings, integers, floats, and booleans. Lists, nested dicts, and arbitrary objects are not supported by most vector stores and will either be silently dropped or will raise an error at insert time. The constraint is annoying but manageable; you flatten anything structured into multiple top-level keys.

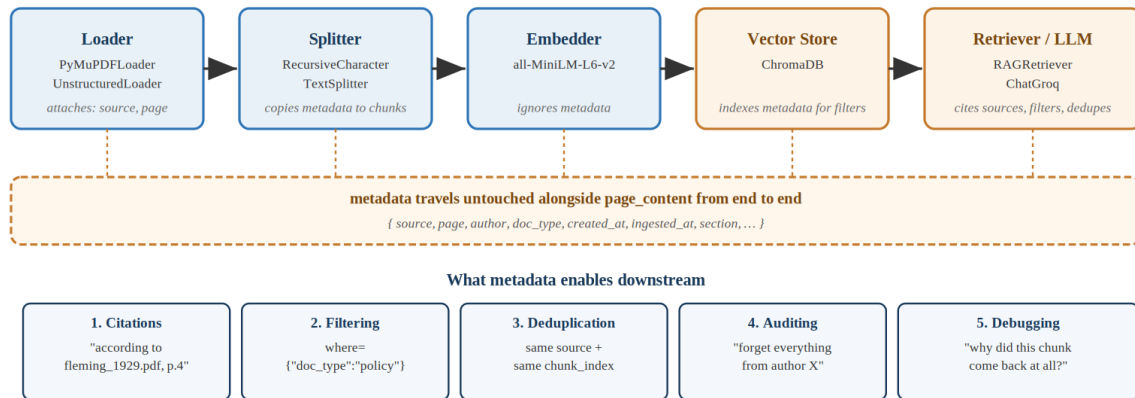
Crucially, metadata is not retrieved by similarity. It is retrieved by exact match or numerical comparison. ChromaDB calls these where filters; the SQL analogy is the WHERE clause. You can ask the store for chunks where doc_type equals “policy” and ingested_at is greater than 1735689600. You cannot ask the store for chunks whose metadata is semantically similar to anything — that operation does not exist, because metadata was never embedded.

Definition — Metadata filter

A boolean predicate evaluated against the metadata dictionary at retrieval time. The vector store first applies the filter to narrow the candidate set, then runs similarity search only over the survivors. This is dramatically more efficient than filtering after retrieval, because the filtered set is smaller before vector math runs.

The Metadata Journey Through the RAG Pipeline

A single dictionary field that surfaces in five different downstream decisions



Metadata is silent during embedding and search, then becomes loud in every layer the user actually sees.

Figure 4.2 — The metadata journey through the RAG pipeline. metadata is silent and invisible during embedding and similarity search, but becomes the central currency of every layer the user actually sees — citations, filtering, deduplication, auditing, and debugging.

Why metadata earns its keep

If metadata cannot be searched semantically, why bother with it at all? Because every concrete need a real RAG system has — beyond “give me the closest text” — is a metadata need. Five examples make this concrete:

- **Citations.** The user is told the answer came from fleming_1929.pdf, page 4. That sentence is impossible to construct unless source and page are sitting in the metadata of every chunk the retriever returned.
- **Filtering.** A finance assistant must answer regulatory questions only from policy documents, never from internal email. The where clause where={ "doc_type": "policy" } is what enforces that, and it works only because doc_type was set during ingestion.
- **Deduplication.** When two near-identical chunks come back, you compare their (source, chunk_index) tuple to decide whether they are the same passage embedded twice or two genuinely different passages that happen to be similar.
- **Auditing and the right to forget.** A user requests deletion of every document they authored, or a regulator requires the removal of a specific data source. Without an author or source field on every chunk, that operation is structurally impossible.
- **Debugging.** A weird chunk surfaces during retrieval and you need to know where it came from. metadata is the only place that can tell you. “It is from page 17 of the contract template” is a one-second answer if metadata is rich and a multi-hour grep otherwise.

page_content is the muscle. metadata is the nervous system. The muscle is what does the lifting, but without the nervous system, the muscle has no idea what it is being asked to do, where it is, or whether the lift was successful. Both are mandatory; neither is optional. Treat them as siblings of equal standing, not as content plus an afterthought.

4.3 Enriching Metadata (source, page, timestamp, author, doc-type)

Loaders give you a starting point — usually source and page — and stop there. That is enough to cite a document but nothing more. To turn metadata into the dependable connective tissue described in Section 4.2, you need to enrich it with several additional fields, deliberately, in a single pass right after loading and before splitting. This section walks through the five enrichment layers in the order they should run, and ends with a skeleton template you can drop into your own ingestion script.

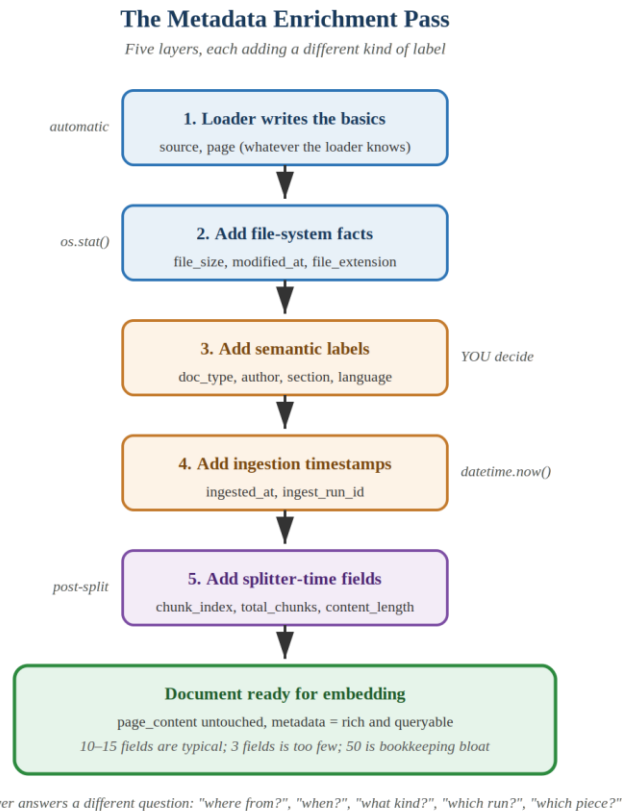


Figure 4.3 — The five layers of metadata enrichment. Each layer answers a different question, and each runs at a specific point in the ingestion pipeline. Skipping a layer is the same as skipping that question forever.

Layer 1 — What the loader already gave you

Most LangChain loaders attach two keys automatically: source (the file path or URL) and, for paginated formats, page (a zero- or one-indexed page number). Verify what your loader produces before you do anything else; the conventions vary. PyPDFLoader uses zero-indexed pages, PyMuPDFLoader uses one-indexed pages, and a simple TextLoader has no page concept at all. Knowing exactly which keys are present saves an hour of confusion later.

```
# Inspect what the loader actually produced
from langchain_community.document_loaders import PyMuPDFLoader

docs = PyMuPDFLoader("../data/pdfs/fleming_1929.pdf").load()
print(docs[0].metadata)
# {'source': '../data/pdfs/fleming_1929.pdf', 'page': 0, ...}
```

Layer 2 — File-system facts

The operating system already knows several useful things about every file on disk: its size, the last time it was modified, its extension. Pull these in once with a single `os.stat` call. They cost nothing to add and

give the retriever the ability to answer queries like “the most recent version of” or “documents larger than 100 KB,” both of which are common in production.

```
from pathlib import Path
from datetime import datetime, timezone

def file_facts(path: str) -> dict:
    p = Path(path)
    st = p.stat()
    return {
        "file_size": st.st_size,
        "modified_at": datetime.fromtimestamp(
            st.st_mtime, tz=timezone.utc
        ).isoformat(),
        "file_extension": p.suffix.lower(),
        "file_name": p.name,
    }
```

Layer 3 — Semantic labels you assign

These are the labels no loader can infer for you because they encode the meaning your project assigns to the document, not facts the file itself carries. `doc_type` is the most important: it is the field your retriever will use most often to filter. Pick a small, fixed vocabulary — policy, manual, email, contract, paper, blog — and use it consistently. A free-form `doc_type` field where one document is tagged “policy” and another “policies” is functionally two different categories from a filter’s point of view. Other useful semantic fields include author, section (the chapter or heading the chunk came from), and language.

Common pitfall — Free-form `doc_type` values

ChromaDB metadata filters are exact-match. “Policy”, “policy”, and “policies” are three distinct values, and a filter for one will silently miss the others. Define your `doc_type` vocabulary as a constant at the top of your ingestion script, then route every document to one of those values. Treat the vocabulary the way you treat an enum.

Layer 4 — Ingestion timestamps

Stamp every chunk with the moment it entered the index, and tag every ingestion run with a unique identifier. The first lets you answer “which chunks have been added since last Friday?” The second lets you cleanly delete an entire run’s worth of chunks if it turns out the ingestion configuration was wrong. Without these two fields, the only way to roll back a bad ingestion is to wipe the entire collection and start over.

```
import uuid
from datetime import datetime, timezone

INGEST_RUN_ID = str(uuid.uuid4()) # generated once per run
INGESTED_AT = datetime.now(timezone.utc).isoformat()

run_facts = {
    "ingest_run_id": INGEST_RUN_ID,
    "ingested_at": INGESTED_AT,
}
```

Layer 5 — Splitter-time fields

After splitting, each chunk inherits its parent Document's metadata wholesale. That is convenient but incomplete: the chunk's position within its parent is now lost unless you add it back. Two fields fix this — `chunk_index` (the chunk's ordinal position within its source document) and `total_chunks` (how many chunks that document was split into). They are the keys that let you reconstruct surrounding context, group adjacent chunks for re-ranking, and detect duplicates emitted by repeated ingestion.

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000, chunk_overlap=200
)

def split_with_indices(docs):
    out = []
    for parent in docs:
        chunks = splitter.split_documents([parent])
        for i, c in enumerate(chunks):
            c.metadata["chunk_index"] = i
            c.metadata["total_chunks"] = len(chunks)
            c.metadata["content_length"] = len(c.page_content)
            out.append(c)
    return out
```

Putting all five layers together — a skeleton template

The five layers compose into a single function. The shape below is intentionally minimal so you can extend it; it is not a finished implementation but a scaffold you fill in with the loaders and vocabularies your project actually uses.

```
# enrich.py – skeleton
from pathlib import Path
from datetime import datetime, timezone
import uuid

INGEST_RUN_ID = str(uuid.uuid4())

DOC_TYPE_BY_DIR = {          # your project's vocabulary
    "policies": "policy",
    "manuals": "manual",
    "papers": "paper",
    "emails": "email",
}

def infer_doc_type(path: Path) -> str:
    for parent in path.parents:
        if parent.name in DOC_TYPE_BY_DIR:
            return DOC_TYPE_BY_DIR[parent.name]
    return "unknown"

def enrich(doc, path: Path) -> None:
    """Mutate doc.metadata in place with all five enrichment layers."""
    md = doc.metadata
    # Layer 2 – file-system facts
    st = path.stat()
    md["file_size"] = st.st_size
```



```

md["modified_at"]    = datetime.fromtimestamp(
                        st.st_mtime, tz=timezone.utc
                    ).isoformat()
md["file_extension"] = path.suffix.lower()
md["file_name"]      = path.name
# Layer 3 – semantic labels
md.setdefault("doc_type", infer_doc_type(path))
md.setdefault("author",   "unknown")
md.setdefault("language", "en")
# Layer 4 – ingestion timestamps
md["ingest_run_id"] = INGEST_RUN_ID
md["ingested_at"]   = datetime.now(timezone.utc).isoformat()
# Layer 5 (chunk_index / total_chunks / content_length)
# is added in the splitting pass, not here.

```

Ten to fifteen metadata fields per chunk is the typical range in a healthy RAG system. Three is too few — you will discover what you are missing only when a downstream feature breaks. Fifty is bookkeeping bloat; it slows inserts, complicates filters, and crowds the index for no marginal benefit. Aim for a deliberate dozen, picked because each one answers a question the system will actually be asked.

4.4 Why Clean Metadata Pays Dividends Downstream

The argument for spending an extra fifteen minutes enriching metadata at ingestion is the same argument for naming variables clearly, writing docstrings, or commenting your config files: it is small work today that prevents large work later. The return on that investment shows up in five distinct places, each of which is worth examining concretely so the abstract claim — “clean metadata pays dividends” — earns its place in your habits.

Dividend 1 — Trustworthy citations

An LLM answer without a source is folklore. An LLM answer with “fleming_1929.pdf, page 4” attached is verifiable. The user can open the file, find the page, and confirm. That ability turns the system from a black-box oracle into an auditable assistant, which is the difference between a demo and a deployment. Citations are nothing more than metadata, formatted into a readable line. Skip metadata at ingestion and you have skipped citations forever.

Dividend 2 — Pre-filtered retrieval

Embedding-only retrieval is a blunt instrument. Adding a where filter on doc_type or author or modified_at sharpens it dramatically. Suppose your corpus contains both legal contracts and casual blog posts, and a user asks a contract-specific question. A pure similarity search will happily mix the two, returning blog text that uses the same words but means something completely different. A two-line metadata filter eliminates that contamination at zero cost — and is impossible without rich metadata.

Dividend 3 — Cheap deduplication

RAG systems re-ingest documents constantly: a user uploads the same PDF twice, a folder watcher fires on a saved-as copy, a colleague drops the same file in a slightly different directory. Without metadata, deduplicating these is an embedding-distance problem — slow, fuzzy, and never quite right. With metadata, it is a one-line set comparison on (source, chunk_index, modified_at). The two paths are not in the same league.

Dividend 4 — Auditability and selective forgetting

Eventually someone will ask the system to forget something. A user invokes their right to deletion. A document turns out to be confidential. An ingestion run was misconfigured and produced bad chunks across thousands of documents. Without metadata, the only way to satisfy any of these is to drop the entire collection and re-ingest from scratch. With metadata, every one of them is a single delete-where call. This is a non-negotiable property in any system that handles personal or regulated data.

Dividend 5 — Faster debugging

When the retriever returns a chunk that looks wrong — and it will, regularly — the question “where did this come from?” needs an immediate, exact answer. Rich metadata answers it in one line: source, page, section, ingest_run_id. You go straight to the original document, look at it with your own eyes, and decide whether the chunk is genuinely irrelevant, mis-tagged, or a casualty of a chunking mistake. Sparse metadata leaves you guessing — and at scale, guessing is how data-quality bugs ferment for months.

Rule of thumb — If you are not sure whether a field belongs in metadata, add it

The marginal cost of an extra metadata key is essentially zero in storage, microseconds in insert time, and absolutely nothing in retrieval cost when no filter touches it. The marginal cost of needing a field that is not there is a re-ingestion of the entire corpus. The asymmetry argues for inclusion. Be deliberate, but lean toward more rather than less.

The Document object is small, but everything in this book passes through it. Build it carefully now, and the next nine chapters of the ingestion pipeline — loading, splitting, embedding, indexing, retrieving — will inherit the discipline. Build it carelessly, and each of those chapters will quietly compound the cost of your earlier shortcut. Chapter 5 begins the loaders, and you are now ready to make sure every Document they produce arrives at the splitter with everything it needs.